Understanding the call tree of ViewManager.cs

2/22/26

Start() method:

Change To Main Menu() is called.

ChangeToMainMenu():

Calls the Coroutine TransitionToState()

TransitionToState():

Sets the Previous State to the current state, & updates the current state.

Calls HandleStateChange().

HandleStateChange():

Deactivates the old UI.

Runs a Switch-case on the current State to update the UI, set "Pausable" bool, and update the time scale.

Game State Manager :

Has the InputManager.CS Script attached.

InputManager.CS:

Calls Update() once per frame.

Update() in InputManager:

Checks for Pause Key presses from Input map reference.

Pause.Invoke() calls OnPause() in ViewManager.CS.

OnPause in ViewManager:

First does some logic to see if it should switch back to Puzzle or exploration Mode. Otherwise, if it is Pausable, call ChangeToPaused().

ChangeToPaused() in ViewManager:

Calls TransitionToState()

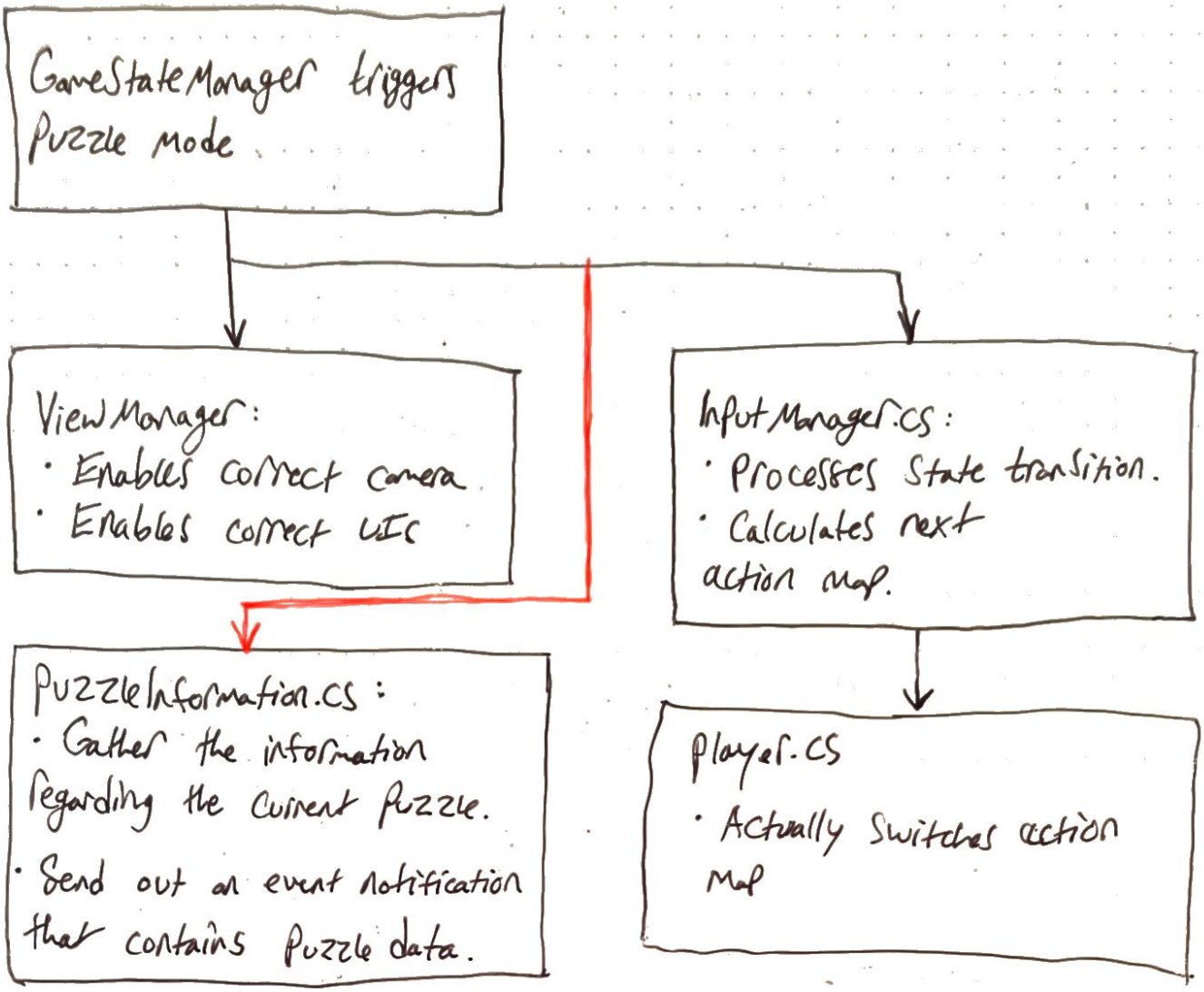
TransitionToState():

Updates variables and calls HandleStateChange().

HandleStateChange():

Updates current UI, Sets "Pausable" to true, Sets timeScale to 0.0f.

"Pausable" also counts for if the game can be unpaused.

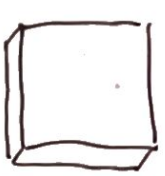


But ViewManager.cs cannot adjust the puzzle camera & UIs accordingly without information from PuzzleInformation.cs

perhaps, then, the event fired by the puzzle terminal must first notify PuzzleInformation.cs before notifying the GameStateManager?

GameStateManager.prefab

- PlayerCamera - good
- DialogueCamera - good
- PuzzleCamera - Changes based on the puzzle
- PlayerUI - good
- DialogueUI - good
- PuzzleUI - Changed based on Puzzle
- PuzzleInformation - null at start but can be initialized anytime we enter a new puzzle.



a bunch of tiles are grouped together & positioned correctly either on Start() or Awake().

InteractablePillar.cs:

This can have a connection to the PuzzleInformation.cs object of a given puzzle.